

INTERVIEW PREPARATION SERIES

LeetCode Practice Set

Complexity & Sorting

10 Hand-Picked Problems for Engineers with ~3 Years of Experience

HOW TO USE THIS DOCUMENT

Every **problem title** and **LEETCODE NNN** reference in this document is **clickable** — click any of them to open the problem on leetcode.com and start practicing.

PART 01

Complexity & Sorting Fundamentals (The “Must-Haves”)

These are the absolute minimum. If you cannot solve these in your sleep, you are not ready.

PROBLEM 01 • [LEETCODE 912](#)

Sort an Array ↗

The Scenario: The interviewer hands you a blank screen. “Implement a sorting function.”

The Catch: You cannot use `Arrays.sort()` or `list.sort()`.

What they test: Can you actually write Merge Sort ($O(n \log n)$ time, $O(n)$ space) or Quick Sort ($O(n \log n)$ time, $O(\log n)$ space) from scratch? They expect you to discuss the trade-offs between stability, memory, and worst-case performance.

Follow-up: “How would you handle negative numbers with Counting Sort?”

PROBLEM 02 • **LEETCODE 88****Merge Sorted Array** ↗

The Scenario: You have two sorted arrays. Merge them into the first array.

The Catch: You cannot use extra space. You must fill from the back to the front.

What they test: Understanding of the Merge Step of Merge Sort. Specifically, Two Pointers technique and in-place manipulation $O(1)$ space.

Real-world context: This is exactly how large data pipelines merge sorted logs.

PROBLEM 03 • **LEETCODE 153****Find Minimum in Rotated Sorted Array** ↗

The Scenario: A sorted array was “rotated” (e.g., $[0, 1, 2, 3] \rightarrow [3, 0, 1, 2]$). Find the smallest number.

The Catch: Do not use a linear scan ($O(n)$). Solve it in $O(\log n)$.

What they test: Applied Binary Search on a non-traditional array.

Insight for 3 YOE: You should immediately recognize this as a binary search variation and explain why the sorted half check works.

PART 02**The “Campus Favorites” (Service & Product Companies)**

Service companies (Wipro, Infosys, TCS) love these because they test logic without needing advanced math.

PROBLEM 04 • **LEETCODE 283****Move Zeroes** ↗

The Scenario: Given an array $[0, 1, 0, 3, 12]$, move all zeros to the end while keeping the relative order of non-zero elements.

The Catch: Do this in-place ($O(1)$ space) and minimize total operations.

What they test: Two Pointers (Slow/Fast pointer) technique. This is incredibly common in Wipro and TCS coding rounds.

PROBLEM 05 • LEETCODE 167

Two Sum II — Input Array Is Sorted ↗

The Scenario: Find two numbers in a sorted array that add to a target.

The Catch: You cannot use a Hash Map (which uses $O(n)$ space). Use $O(1)$ space.

What they test: The classic Two Pointers approach (left and right). This is a staple for 3 YOE interviews to see if you over-engineer a solution.

PROBLEM 06 • LEETCODE 278

First Bad Version ↗

The Scenario: You have n versions `[1,2,...,n]` and a function `isBadVersion(version)`. Find the first bad version.

The Catch: Minimize the calls to the API.

What they test: Binary Search for edge finding (lower bound).

Real-world context: This mirrors production rollback scenarios.

PART 03

Advanced Sorting & Partitioning (The “Product Company” Differentiators)

If you are aiming for Amazon, Walmart, or a well-funded startup (off-campus), expect these.

PROBLEM 07 • [LEETCODE 215](#)

Kth Largest Element in an Array ↗

The Scenario: Find the kth largest element (e.g., $k=2$ in `[3,2,1,5,6,4]` returns 5).

The Catch: Do not sort the whole array ($O(n \log n)$). Solve it in $O(n)$ average time.

What they test: Quick Select (using the partition logic from Quick Sort).

3 YOE Expectation: You must explain why Quick Select is faster than a heap for this specific case, and discuss the worst-case $O(n^2)$ scenario.

PROBLEM 08 • [LEETCODE 75](#)

Sort Colors ↗

The Scenario: Sort an array containing only 0, 1, 2 (representing red, white, blue).

The Catch: Do not use a counting sort (counting 0s,1s,2s). Do it in one pass ($O(n)$) with $O(1)$ space.

What they test: Dutch National Flag Algorithm (Three-way partitioning). This is the ultimate test of your understanding of the partitioning step of Quick Sort.

PART 04

Recursion & Divide & Conquer (The “Deep Dive”)

PROBLEM 09 • [LEETCODE 21](#)

Merge Two Sorted Lists ↗

The Scenario: Merge two sorted linked lists into one sorted list.

The Catch: Solve it Recursively.

What they test: Understanding of the Recursive Case and Base Case. It is beautiful because the iterative solution is 10 lines, but the recursive solution is 3 lines.

Why here: This tests if you understand the call stack visualization we discussed earlier.

PROBLEM 10 • [LEETCODE 240](#)

Search a 2D Matrix II ↗

The Scenario: A matrix where rows are sorted left-to-right and columns are sorted top-to-bottom. Find a target value.

The Catch: Start from the top-right corner.

What they test: Binary Search Tree logic on a matrix. This is an “Advanced Sorting” thinking problem where you leverage the inherent order of the data.

SUMMARY

Summary Checklist for You

If you have 3 years of experience and an interview next week:

1. Do [LeetCode 912](#) (Merge Sort) until you can write it blindfolded.
2. Do [LeetCode 215](#) (Kth Largest) and master Quick Select.

3. Master the Two Pointers pattern ([LeetCode 88](#), [LeetCode 167](#), [LeetCode 283](#)).
4. Practice the “Rotated Array” binary search ([LeetCode 153](#)).

PRO TIP

When you solve these, verbally state the **Time Complexity (Big-O)** and **Space Complexity** before you write the first line of code. That is the habit that gets 3+ YOE candidates hired over juniors.